cl-freelock API Reference

Andrew D. France

May 27, 2026

Contents

1 Getting Started: Installation

Once available on quicklisp, you will be able to simply run:

```
(ql:quickload :cl-freelock)
```

For now, clone the repository into your Quicklisp local-projects directory:

```
cd ~/quicklisp/local-projects/
git clone https://github.com/ItsMeForLua/cl-freelock.git
```

Then, load the system in your REPL:

```
(ql:quickload :cl-freelock)
```

2 Basic Usage

In this section, we will go over the basic usage of producer-consumer with unbounded queue, bounded buffers, SPSC, error handling, and atomic operations.

2.1 Producer-Consumer with Unbounded Queue

The algorithm used for unbounded queues is based off of the Michael-Scott algorithm. We specifically use Compare-And-Swap (CAS) operations to link nodes and swing the head and tail pointers.

We identify three main reasons one would use this particular queue type:

1. When the workload is highly variable in predictability, such that you cannot guarantee a maximum capacity for your queue ahead of time.

2. You have a thread pool architecture with multiple producer threads that are throwing jobs onto the queue, and multiple worker threads are simultaneously pulling jobs off. i.e., An N-to-N architecture.

3. If your project requires immunity from deadlocks.

Producers

A producer thread works by calling `(queue-push *work-queue* item)`. Because the producer thread is lock-free, the procedure will never block a thread.

An example of defining a producer:

```
(defun producer ()
(loop for i from 1 to 1000 do
(queue-push *work-queue* i)
(sleep 0.001)))
```

n.b., It is very important you consider the reasoning for adding a sleep in this example. The sleep is just to simulate the time it may take to fetch the next item. It is not needed in the procedure within normal use-cases; however, if you simply ran the example in a fresh REPL, you would risk locking up your entire environment.[1, 2]

Consumers

A consumer thread works by calling `(queue-pop *work-queue*)`. This procedure does not return lisp conditions; instead, `queue-pop` returns two specific values:

1. The `item` itself or `NIL` if empty.

2. A `success` boolean, `T` if an item was grabbed, `NIL` if the queue was empty.

1. This is called "CPU Starvation", which is when a procedure consumes 100% of a systems CPU core.
2. Concurrent programming is a matter of managing the interleaving of different processes as they interact with a shared state – hence, if we ran a loop from i to 1000 without sleep, most modern CPUs would actually be so fast that the producer may finish pushing all 1000 items before a consumer thread has a chance to wakeup.

n.b., If a queue happens to be empty, then the consumer thread should handle it gracefully by briefly sleeping before polling again. The sleep is a small number, which you set to prevent busy waiting.[3]

You must wrap the procedure in a `multiple-value-bind` to check if the pop was successful.

```
(defparameter *work-queue* (make-queue))
;; Producer thread
(defun producer ()
(loop for i from 1 to 1000 do
(queue-push *work-queue* i)
(sleep 0.001)))

;; Consumer thread
(defun consumer ()
(loop
(multiple-value-bind (item found)
(queue-pop *work-queue*)
(if found
(format t "Processing: ~A~%" item)
(sleep 0.001)))))
```

2.2 Bounded Buffer

```
(defparameter *buffer* (make-bounded-queue 64))  ; Must be power of 2

;; Non-blocking operations
(when (bounded-queue-push *buffer* "data")
(format t "Push succeeded~%"))

(multiple-value-bind (value success)
(bounded-queue-pop *buffer*)
(when success
(format t "Got: ~A~%" value)))
```

3. Busy waiting (or spinning) is when a consumer constantly asks an empty queue if it has any data in queue – consequently, this tanks a systems performance.

2.3 SPSC

```
(defparameter *spsc* (make-spsc-queue 256))  ; Must be power of 2
;; Producer thread only
(when (spsc-push *spsc* "fast-data")
(format t "Pushed successfully~%"))
;; Consumer thread only
(multiple-value-bind (value success)
(spsc-pop *spsc*)
(when success
(format t "Got: ~A~%" value)))
```

2.4 Error Handling

Operations return multiple values to indicate success/failure rather than signaling conditions:

```
;; Always check the second return value
(multiple-value-bind (value success)
(queue-pop *queue*)
(if success
(process value)
(handle-empty-queue)))
```

2.5 Atomic Operations

Types

2.5.1 ATOMIC-REF

Functions

2.5.2 MAKE-ATOMIC-REF

2.5.3 ATOMIC-REF-VALUE

2.6 Macros

2.6.1 CAS

2.6.2 ATOMIC-INCF

2.7 Utility Functions

2.7.1 MEMORY-BARRIER

2.8 Unbounded Queue

Class

2.8.1 QUEUE

Lock-free unbounded queue supporting multiple producers and consumers.

Functions

2.8.2 MAKE-QUEUE

2.8.3 QUEUE-PUSH

2.8.4 QUEUE-POP

2.8.5 QUEUE-EMPTY-P

2.9 Bounded Queue

Class

2.9.1 BOUNDED-QUEUE

Functions

2.9.2 MAKE-BOUNDED-QUEUE

2.9.3 BOUNDED-QUEUE-CAPACITY

2.9.4 BOUNDED-QUEUE-PUSH

2.9.5 BOUNDED-QUEUE-POP

2.9.6 BOUNDED-QUEUE-EMPTY-P

2.9.7 BOUNDED-QUEUE-FULL-P

Batch Operations

2.9.8 BOUNDED-QUEUE-PUSH-BATCH

2.9.9 BOUNDED-QUEUE-POP-BATCH

2.10 SPSC Queue

Class

2.10.1 SPSC-QUEUE

Functions

2.10.2 MAKE-SPSC-QUEUE

3.1 General Performance Principles

3.1.1 Choose the Right Data Structure

3.1.2 Capacity Selection

3.1.3 Thread Contention

3.2 Optimization Techniques

3.2.1 Batch Operations

3.2.2 Avoid Busy Waiting

3.2.3 Memory Allocation

3.3 Implementation-Specific Notes

3.3.1 SBCL

3.3.2 CLASP

3.4 Common Pitfalls

3.4.1 Wrong capacity

3.4.2 Thread safety

3.4.3 Busy waitings

3.4.4 Memory leaks

3.4.5 False sharing

3.5 Recommended Patterns

3.5.1 Producer-Consumer Pool

3.5.2 Basic Producer-Consumer

3.5.3 Bounded Buffer with Batch Processing

3.5.4 SPSC Pipeline